

2.5 데이터 전처리(Data Preprocessing)

목차

1. 데이터 전처리 개요
2. 데이터 인코딩
 1. 레이블 인코딩(Label Encoding)
 2. 인코딩 클래스 확인과 디코딩
 3. 레이블 인코딩의 한계
 4. 원-핫 인코딩(One-Hot Encoding)
 5. 판다스 `get_dummies()`
3. 피처 스케일링과 정규화
 1. 스케일링이 필요한 이유
 2. `StandardScaler`
 3. `MinMaxScaler`
4. 학습/테스트 데이터의 스케일링 변환 시 유의사항
 1. 실습용 데이터 생성
 2. 잘못된 사례 - 테스트 데이터에 `fit()` 재호출
 3. 올바른 사례 - `transform()`만 호출
5. 핵심 요약 및 머신러닝 활용 포인트

1. 데이터 전처리 개요

머신러닝 알고리즘만큼 중요한 것이 **데이터 전처리**입니다. **Garbage In, Garbage Out**이라는 말처럼, 아무리 뛰어난 알고리즘도 잘못된 데이터로는 좋은 결과를 낼 수 없습니다.

사이킷런의 머신러닝 알고리즘을 적용하기 전에 반드시 처리해야 할 사항은 다음과 같습니다.

처리 항목	내용
결손값(NaN, Null) 처리	허용되지 않으므로 고정값이나 평균값으로 대체 하거나 해당 피처를 삭제해야 합니다. Null이 매우 많은 피처는 삭제를, 일부라면 대체를 고려합니다.
문자열 값 처리	사이킷런은 문자열 값을 입력값으로 허용하지 않습니다 . 모든 문자열은 인코딩되어 숫자형 으로 변환되어야 합니다.
불필요한 피처 제거	주민번호, 아이디 같은 식별자 성격의 피처 는 예측에 중요하지 않으므로 삭제하는 것이 좋습니다.
피처 스케일링	피처마다 값의 범위가 크게 다르면 특정 피처가 과도한 영향을 주므로 범위를 맞춰 줍니다.

■ 문자열 피처의 두 유형

- **카테고리형**: 성별, 등급, 지역 등 **범주를 나타내는 값**. → 코드값으로 **인코딩**합니다.
- **텍스트형**: 상품 설명, 리뷰 등 **자유로운 문장**. → 피처 벡터화(TF-IDF 등)를 하거나, 불필요하면 삭제합니다.

2. 데이터 인코딩

머신러닝을 위한 대표적인 인코딩 방식은 **레이블 인코딩**과 **원-핫 인코딩** 두 가지입니다.

2.1 레이블 인코딩(Label Encoding)

레이블 인코딩은 카테고리 피처를 **코드형 숫자값으로 변환**하는 것입니다.

```
from sklearn.preprocessing import LabelEncoder

items=['TV', '냉장고', '전자레인지', '컴퓨터', '선종기', '선종기', '믹서', '믹서']

# LabelEncoder를 객체로 생성한 후, fit() 과 transform() 으로 label 인코딩 수행.
encoder = LabelEncoder()
encoder.fit(items)
labels = encoder.transform(items)
print('인코딩 변환값:', labels)
```

줄	코드	상세 설명
1	from sklearn.preprocessing import LabelEncoder	<code>sklearn.preprocessing</code> 모듈에서 <code>LabelEncoder</code> 클래스를 임포트합니다.
3	items=['TV', '냉장고', ..., '믹서']	8개의 상품명 문자열 리스트입니다. 고유값은 TV, 냉장고, 전자레인지, 컴퓨터, 선종기, 믹서 의 6가지이며, 선종기와 믹서는 2번씩 등장합니다.
6	encoder = LabelEncoder()	인코더 객체 를 생성합니다. 이 시점에는 아무것도 학습되지 않은 상태입니다.
7	encoder.fit(items)	<code>fit()</code> 은 데이터를 변환하지 않습니다 . 어떤 고유값들이 있는지 파악해 알파벳·가나다순으로 정렬한 매핑 규칙 을 학습할 뿐입니다.

8	<code>labels = encoder.transform(items)</code>	transform()이 실제 변환을 수행합니다. fit에서 만든 매핑에 따라 각 문자열을 숫자로 바꿔 ndarray로 반환합니다.
9	<code>print('인코딩 변환값:', labels)</code>	결과는 <code>[0 1 4 5 3 3 2 2]</code> 입니다. TV→0, 냉장고→1, 전자레인지→4, 컴퓨터→5, 선풍기→3, 믹서→2로 변환되었습니다. 같은 값(선풍기, 믹서)은 항상 같은 숫자 가 됩니다.

▶ 실행 결과

```
인코딩 변환값: [0 1 4 5 3 3 2 2]
```

■ 숫자 부여 기준

등장 순서가 아니라 **정렬 순서**로 번호가 매겨집니다. 영문은 알파벳순, 한글은 가나다순입니다. 그래서 'TV'가 0, 그다음 가나다순으로 '냉장고'(1), '믹서'(2), '선풍기'(3), '전자레인지'(4), '컴퓨터'(5)가 됩니다. `fit_transform( )` 을 쓰면 두 단계를 한 줄로 처리할 수 있습니다.

2.2 인코딩 클래스 확인과 디코딩

```
print('인코딩 클래스:', encoder.classes_)
```

줄	코드	상세 설명
1	<code>encoder.classes_</code>	학습된 매핑 정보 를 담은 속성입니다. 배열의 인덱스가 곧 인코딩된 숫자 이고 값이 원본 문자열입니다. 즉 <code>classes_[0]='TV'</code> 이므로 TV가 0입니다. 언더스코어(_)로 끝나는 것은 fit 이후에 생성되는 속성 이라는 사이킷런의 관례입니다.

▶ 실행 결과

```
인코딩 클래스: ['TV' '냉장고' '믹서' '선풍기' '전자레인지' '컴퓨터']
```

```
print('디코딩 원본 값:', encoder.inverse_transform([4, 5, 2, 0, 1, 1, 3, 3]))
```

줄	코드	상세 설명
1	<code>encoder.inverse_transform([4,5,2,0,1,1,3,3])</code>	인코딩의 역변환 입니다. 숫자를 다시 원본 문자열로 되돌립니다. 4→전자레인지, 5→컴퓨터, 2→믹서 순으로 변환됩니다. 모델의 예측 결과를 사람이 읽을 수 있는 형태로 되돌릴 때 매우 유용합니다.

▶ 실행 결과

```
디코딩 원본 값: ['전자레인지' '컴퓨터' '믹서' 'TV' '냉장고' '냉장고' '선풍기' '선풍기']
```

2.3 레이블 인코딩의 한계

⚠ 숫자에 없던 크기 관계가 생긴다

레이블 인코딩은 문자를 숫자로 바꾸지만, 그 과정에서 **원래 없던 대소 관계와 순서가 생겨 버립니다**. 냉장고(1)보다 컴퓨터(5)가 **5배 크거나 우수하다는 의미는 전혀 없는데**, 알고리즘은 숫자를 그대로 받아들여 가중치를 다르게 부여합니다.

따라서 **선형 회귀, 로지스틱 회귀, SVM 같은 알고리즘에는 레이블 인코딩을 적용하면 안 됩니다**. 반면 **결정 트리 계열** (Decision Tree, Random Forest, GBM 등)은 숫자의 크기를 그대로 해석하지 않고 **분기 기준으로만** 사용하므로 레이블 인코딩을 써도 무방합니다.

2.4 원-핫 인코딩(One-Hot Encoding)

원-핫 인코딩은 **고유값의 개수만큼 새로운 컬럼을 만들고**, 해당하는 컬럼에만 1을 표시하고 나머지는 0으로 채우는 방식입니다. 이렇게 하면 **숫자의 크기 관계 문제가 사라집니다**.

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

items=['TV', '냉장고', '전자레인지', '컴퓨터', '선종기', '선풍기', '믹서', '믹서']

# 2차원 ndarray로 변환합니다.
items = np.array(items).reshape(-1, 1)

# 원-핫 인코딩을 적용합니다.
oh_encoder = OneHotEncoder()
oh_encoder.fit(items)
oh_labels = oh_encoder.transform(items)

# OneHotEncoder로 변환한 결과는 희소행렬이므로 toarray()를 이용해 밀집 행렬로 변환.
print('원-핫 인코딩 데이터')
print(oh_labels.toarray())
print('원-핫 인코딩 데이터 차원')
print(oh_labels.shape)
```

줄	코드	상세 설명
7	<code>items = np.array(items).reshape(-1, 1)</code>	매우 중요한 단계 입니다. OneHotEncoder는 2차원 데이터 만 입력받으므로, 리스트를 ndarray로 바꾼 뒤 <code>reshape(-1, 1)</code> 로 (8, 1) 형태의 2차원 으로 만들어야 합니다. 이 과정을 빠뜨리면 오류가 발생합니다.
10	<code>oh_encoder = OneHotEncoder()</code>	원-핫 인코더 객체를 생성합니다.
11	<code>oh_encoder.fit(items)</code>	고유값 6개를 파악해 컬럼 구성을 학습 합니다.
12	<code>oh_labels = oh_encoder.transform(items)</code>	실제 변환을 수행합니다. 반환값은 일반 배열이 아니라 희소 행렬(sparse matrix) 입니다. 원-핫 결과는 대부분이 0이므로 0이 아닌 위치만 저장해 메모리를 절약하는 형식입니다.
16	<code>oh_labels.toarray()</code>	희소 행렬을 사람이 볼 수 있는 밀집 행렬(일반 ndarray) 로 변환합니다. 이 과정을 거치지 않으면 내용이 아니라 행렬 요약 정보만 출력됩니다.
18	<code>oh_labels.shape</code>	(8, 6) → 8개 행 (원본 데이터 수)과 6개 열 (고유값 수)입니다. 컬럼이 1개에서 6개로 늘어났습니다 .

▶ 실행 결과

```
원-핫 인코딩 데이터
[[1. 0. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0. 0.]
 [0. 0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0. 0.]]
원-핫 인코딩 데이터 차원
(8, 6)
```

■ 결과 해석

컬럼 순서는 `classes_` 와 같은 ['TV','냉장고','믹서','선풍기','전자레인지','컴퓨터']입니다. 첫 행은 'TV'이므로 0번 위치만 1이고, 세 번째 행은 '전자레인지'이므로 4번 위치만 1입니다. 각 행에는 정확히 하나의 1만 존재해서 '원-핫(One-Hot)'이라는 이름이 붙었습니다.

⚠ 원-핫 인코딩의 부작용

고유값이 많으면 컬럼 수가 폭발적으로 증가합니다(차원의 저주). 예를 들어 우편번호처럼 값이 수천 개인 피처를 원-핫 인코딩 하면 컬럼이 수천 개가 되어 메모리와 학습 시간이 급증하고 성능도 떨어질 수 있습니다. 이럴 때는 값을 상위 몇 개로 묶거나 다른 인코딩 기법을 검토해야 합니다.

⚠ 버전 차이 - `sparse_output` 파라미터

사이킷런 1.2부터 `sparse` 파라미터가 `sparse_output` 으로 이름이 바뀌었고, 1.4에서 기존 이름은 제거되었습니다. 처음부터 일반 배열로 받고 싶다면 `OneHotEncoder(sparse_output=False)` 를 사용하세요. 구버전 교재의 `sparse=False` 코드는 현재 버전에서 `TypeError`가 발생합니다.

2.5 판다스 `get_dummies()`

판다스는 원-핫 인코딩을 훨씬 간편하게 지원합니다.

```
import pandas as pd

df = pd.DataFrame({'item':['TV','냉장고','전자레인지','컴퓨터','선풍기','선풍기','믹서','믹서']})
pd.get_dummies(df)
```

줄	코드	상세 설명
3	<code>df = pd.DataFrame({'item':[...]})</code>	상품명명을 담은 DataFrame을 만듭니다.
4	<code>pd.get_dummies(df)</code>	숫자 변환 없이 문자열을 바로 넣어도 원-핫 인코딩이 수행됩니다. <code>reshape</code> 도, <code>fit/transform</code> 도 필요 없습니다. 새 컬럼명은 원본컬럼명_값 형태(<code>item_TV</code> 등)로 자동 생성되어 의미를 바로 알 수 있습니다.

▶ 실행 결과

```
item_TV  item_냉장고  item_믹서  item_선풍기  item_전자레인지  item_컴퓨터
0      True      False      False      False      False      False
1     False       True      False      False      False      False
2     False      False      False      False       True      False
3     False      False      False      False      False       True
4     False      False      False       True      False      False
```

5	False	False	False	True	False	False
6	False	False	True	False	False	False
7	False	False	True	False	False	False

⚠ 버전 차이 - 결과 타입이 bool로 변경

pandas 2.0부터 `get_dummies()` 의 결과가 0/1 정수가 아니라 True/False(bool)로 반환됩니다. 사이킷런 모델에 그대로 넣어도 내부적으로 0/1로 처리되어 학습에는 문제가 없습니다. 숫자로 받고 싶다면 `pd.get_dummies(df, dtype=int)` 를 사용하세요.

■ OneHotEncoder vs get_dummies

구분	OneHotEncoder	get_dummies
입력 준비	2차원 변환 필요(<code>reshape</code>)	DataFrame 그대로
사용 편의	fit → transform 2단계	한 줄
컬럼명	별도 확인 필요	자동 생성
매핑 재사용	가능 (학습된 기준을 테스트에 적용)	불가 (호출할 때마다 새로 계산)
권장 상황	모델 파이프라인에 포함할 때	탐색적 분석, 일회성 변환

실무에서는 학습 데이터와 테스트 데이터의 컬럼 구성이 반드시 같아야 하므로, 파이프라인에서는 `OneHotEncoder` 가 안전합니다.

3. 피쳐 스케일링과 정규화

3.1 스케일링이 필요한 이유

피쳐 스케일링(Feature Scaling)은 서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업입니다. 예를 들어 나이(0~100)와 연봉(0~100,000,000)이 함께 있으면, 거리 기반 알고리즘은 숫자가 큰 연봉에만 좌우됩니다.

방식	설명	결과 범위
표준화 (Standardization)	평균이 0, 분산이 1인 가우시안 정규 분포를 가진 값으로 변환 $(x - \text{평균}) / \text{표준편차}$	제한 없음 (대부분 -3 ~ 3)
정규화 (Normalization)	서로 다른 피쳐의 크기를 0~1 사이로 통일 $(x - \text{최솟값}) / (\text{최댓값} - \text{최솟값})$	0 ~ 1

■ 스케일링이 특히 중요한 알고리즘

SVM, 선형 회귀, 로지스틱 회귀는 데이터가 가우시안 분포를 가진다고 가정하므로 표준화가 성능에 큰 영향을 줍니다. K-Means, KNN 등 거리 기반 알고리즘도 스케일링이 필수입니다. 반면 결정 트리 계열은 분기 기준만 사용하므로 스케일링이 거의 영향을 주지 않습니다.

3.2 StandardScaler

먼저 변환 전 붓꽃 데이터의 평균과 분산을 확인합니다.

```

from sklearn.datasets import load_iris
import pandas as pd
# 붓꽃 데이터 셋을 로딩하고 DataFrame으로 변환합니다.
iris = load_iris()
iris_data = iris.data
iris_df = pd.DataFrame(data=iris_data, columns=iris.feature_names)

print('feature 들의 평균 값')
print(iris_df.mean())
print('\nfeature 들의 분산 값')
print(iris_df.var())

```

줄	코드	상세 설명
6	iris_df = pd.DataFrame(data=iris_data, columns=iris.feature_names)	ndarray를 DataFrame으로 변환해 통계를 보기 쉽게 만듭니다.
9	iris_df.mean()	컬럼별 평균 입니다. 5.84, 3.06, 3.76, 1.20으로 제각각 입니다.
11	iris_df.var()	컬럼별 분산 입니다. 0.19에서 3.12까지 16배 이상 차이 가 납니다.

▶ 실행 결과

```

feature 들의 평균 값
sepal length (cm)    5.843333
sepal width (cm)     3.057333
petal length (cm)    3.758000
petal width (cm)     1.199333
dtype: float64

```

```

feature 들의 분산 값
sepal length (cm)    0.685694
sepal width (cm)     0.189979
petal length (cm)    3.116278
petal width (cm)     0.581006
dtype: float64

```

```

from sklearn.preprocessing import StandardScaler

# StandardScaler객체 생성
scaler = StandardScaler()
# StandardScaler 로 데이터 셋 변환. fit( ) 과 transform( ) 호출.
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

#transform( )시 scale 변환된 데이터 셋이 numpy ndarry로 반환되어 이를 DataFrame으로 변환
iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)
print('feature 들의 평균 값')
print(iris_df_scaled.mean())
print('\nfeature 들의 분산 값')
print(iris_df_scaled.var())

```

줄	코드	상세 설명
4	scaler = StandardScaler()	표준화 스케일러 객체를 생성합니다.
6	scaler.fit(iris_df)	각 컬럼의 평균과 표준편차를 계산해 기억 합니다. 아직 데이터는 바뀌지 않습니다.

7	<code>iris_scaled = scaler.transform(iris_df)</code>	기억한 평균·표준편차로 $(x - \text{평균}) / \text{표준편차}$ 계산을 수행합니다. 반환값은 DataFrame이 아니라 ndarray 입니다.
10	<code>iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)</code>	ndarray를 다시 DataFrame으로 되돌립니다. 컬럼명이 사라지므로 다시 지정해 주는 것이 중요합니다.
12~15	<code>mean() / var()</code>	변환 후 평균은 0에 가까운 값 , 분산은 1에 가까운 값 이 됩니다.

▶ 실행 결과

```
feature 들의 평균 값
sepal length (cm)    -1.690315e-15
sepal width (cm)     -1.842970e-15
petal length (cm)    -1.698641e-15
petal width (cm)     -1.409243e-15
dtype: float64

feature 들의 분산 값
sepal length (cm)     1.006711
sepal width (cm)      1.006711
petal length (cm)     1.006711
petal width (cm)      1.006711
dtype: float64
```

■ 결과 해석 - 왜 정확히 0과 1이 아닌가

- 평균이 $-1.69e-15$ 처럼 표시되는 것은 **0.000000000000000169**를 의미합니다. 이론적으로는 0이지만 **부동소수점 연산의 미세한 오차**로 완전한 0이 되지 않을 뿐, 사실상 0입니다.
- 분산이 1.0067인 이유는 판다스의 `var()`가 기본적으로 **표본 분산($n-1$ 로 나눔)**을 계산하는 반면, 사이킷런은 **모분산(n 으로 나눔)**을 사용하기 때문입니다. 150개 데이터에서 $150/149 \approx 1.0067$ 의 차이가 발생합니다. `var(ddof=0)`으로 계산하면 정확히 1.0이 나옵니다.
- 네 피처의 분산이 **모두 동일**해졌다는 점이 핵심입니다. 이제 어떤 피쳐도 크기 때문에 과도한 영향을 주지 않습니다.

3.3 MinMaxScaler

```
from sklearn.preprocessing import MinMaxScaler

# MinMaxScaler 객체 생성
scaler = MinMaxScaler()
# MinMaxScaler 로 데이터 셋 변환. fit() 과 transform() 호출.
scaler.fit(iris_df)
iris_scaled = scaler.transform(iris_df)

# transform()시 scale 변환된 데이터 셋이 numpy ndarray로 반환되어 이를 DataFrame으로 변환
iris_df_scaled = pd.DataFrame(data=iris_scaled, columns=iris.feature_names)
print('feature들의 최솟값')
print(iris_df_scaled.min())
print('\nfeature들의 최댓값')
print(iris_df_scaled.max())
```

줄	코드	상세 설명
4	<code>scaler = MinMaxScaler()</code>	0~1 범위로 정규화하는 스케일러입니다. 사용법은 StandardScaler와 완전히 동일 합니다 — 이것이 사이킷런 API의 일관성입니다.

6	<code>scaler.fit(iris_df)</code>	각 컬럼의 최솟값 과 최댓값 을 계산해 기억합니다.
7	<code>iris_scaled = scaler.transform(iris_df)</code>	$(x - \text{최솟값}) / (\text{최댓값} - \text{최솟값})$ 을 적용합니다.
12~15	<code>min() / max()</code>	모든 피처의 최솟값이 정확히 0 , 최댓값이 정확히 1 이 되었습니다.

▶ 실행 결과

```
feature들의 최솟값
sepal length (cm)    0.0
sepal width (cm)     0.0
petal length (cm)    0.0
petal width (cm)     0.0
dtype: float64
```

```
feature들의 최댓값
sepal length (cm)    1.0
sepal width (cm)     1.0
petal length (cm)    1.0
petal width (cm)     1.0
dtype: float64
```

■ StandardScaler vs MinMaxScaler 선택 기준

- **StandardScaler** : 데이터가 **정규 분포에 가깝고** 이상치가 적을 때. 선형 모델·SVM에 적합합니다.
- **MinMaxScaler** : 값의 범위를 **명확히 제한**해야 할 때(이미지 픽셀, 신경망 입력 등). 단, **이상치에 매우 취약**합니다 — 극단 값 하나가 최댓값이 되면 나머지 값들이 0 근처로 몰립니다.
- 이상치가 많다면 중앙값과 사분위수를 쓰는 **RobustScaler** 를 고려하세요.

4. 학습/테스트 데이터의 스케일링 변환 시 유의사항

이 절은 실무에서 가장 많은 실수가 발생하는 부분이므로 특히 주의 깊게 봐야 합니다.

4.1 실습용 데이터 생성

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 학습 데이터는 0 부터 10까지, 테스트 데이터는 0 부터 5까지 값을 가지는 데이터 세트로 생성
# Scaler클래스의 fit(), transform()은 2차원 이상 데이터만 가능하므로 reshape(-1, 1)로 차원 변경
train_array = np.arange(0, 11).reshape(-1, 1)
test_array = np.arange(0, 6).reshape(-1, 1)
```

줄	코드	상세 설명
6	<code>train_array = np.arange(0, 11).reshape(-1, 1)</code>	0~10 의 11개 값을 가진 학습 데이터를 만듭니다. <code>reshape(-1, 1)</code> 로 (11, 1) 2차원 으로 변환합니다 — 스케일러가 2차원만 받기 때문입니다.
7	<code>test_array = np.arange(0, 6).reshape(-1, 1)</code>	0~5 의 6개 값을 가진 테스트 데이터입니다. 학습 데이터와 값의 범위가 다르다 는 점이 이 실습의 핵심입니다.

4.2 잘못된 사례 - 테스트 데이터에 fit() 재호출

```
# MinMaxScaler 객체에 별도의 feature_range 파라미터 값을 지정하지 않으면 0~1 값으로 변환
scaler = MinMaxScaler()

# fit()하게 되면 train_array 데이터의 최솟값이 0, 최댓값이 10으로 설정.
scaler.fit(train_array)

# 1/10 scale로 train_array 데이터 변환함. 원본 10-> 1로 변환됨.
train_scaled = scaler.transform(train_array)

print('원본 train_array 데이터:', np.round(train_array.reshape(-1), 2))
print('Scale된 train_array 데이터:', np.round(train_scaled.reshape(-1), 2))
```

줄	코드	상세 설명
5	scaler.fit(train_array)	학습 데이터의 최솟값 0, 최댓값 10 을 기억합니다.
8	train_scaled = scaler.transform(train_array)	0~10이 0~1로 변환 됩니다. 즉 1/10 스케일 이 적용되어 원본 1은 0.1, 5는 0.5, 10은 1이 됩니다.
11	np.round(train_scaled.reshape(-1), 2)	2차원을 reshape(-1) 로 1차원으로 펼쳐 보기 좋게 출력합니다.

▶ 실행 결과

```
원본 train_array 데이터: [ 0  1  2  3  4  5  6  7  8  9 10]
Scale된 train_array 데이터: [0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]
```

```
# MinMaxScaler에 test_array를 fit()하게 되면 원본 데이터의 최솟값이 0, 최댓값이 5로 설정됨
scaler.fit(test_array)

# 1/5 scale로 test_array 데이터 변환함. 원본 5->1로 변환.
test_scaled = scaler.transform(test_array)

# test_array의 scale 변환 출력.
print('원본 test_array 데이터:', np.round(test_array.reshape(-1), 2))
print('Scale된 test_array 데이터:', np.round(test_scaled.reshape(-1), 2))
```

줄	코드	상세 설명
2	scaler.fit(test_array)	여기가 치명적인 실수입니다. 같은 scaler 객체에 테스트 데이터로 다시 fit() 을 호출하면, 기억하고 있던 학습 데이터의 기준(0~10) 이 지워지고 테스트 데이터 기준(0~5)으로 덮어써집니다.
5	test_scaled = scaler.transform(test_array)	이제 1/5 스케일 이 적용됩니다. 원본 5가 1로 변환됩니다.
8~9	print(...)	결과를 보면 0, 0.2, 0.4, 0.6, 0.8, 1 로 변환되었습니다.

▶ 실행 결과

```
원본 test_array 데이터: [0 1 2 3 4 5]
Scale된 test_array 데이터: [0.  0.2 0.4 0.6 0.8 1. ]
```

⚠ 무엇이 잘못되었는가 - 같은 값이 다른 숫자가 된다

학습 데이터에서 원본 5는 0.5로 변환되었는데, 테스트 데이터에서는 원본 5가 1.0으로 변환되었습니다. 동일한 원본 값이 서로 다른 값으로 스케일링된 것입니다.

모델은 "0.5 = 원본 5"라고 학습했는데 예측 시에는 "1.0 = 원본 5"인 데이터를 받게 되므로, 예측 결과가 완전히 어긋납니다. 학습과 예측의 기준이 달라지는 이 문제는 오류 메시지 없이 조용히 성능만 떨어뜨리기 때문에 더욱 위험합니다.

4.3 올바른 사례 - transform()만 호출

```
scaler = MinMaxScaler()
scaler.fit(train_array)
train_scaled = scaler.transform(train_array)
print('원본 train_array 데이터:', np.round(train_array.reshape(-1), 2))
print('Scale된 train_array 데이터:', np.round(train_scaled.reshape(-1), 2))

# test_array에 Scale 변환을 할 때는 반드시 fit()을 호출하지 않고 transform() 만으로 변환해야 함.
test_scaled = scaler.transform(test_array)
print('\n원본 test_array 데이터:', np.round(test_array.reshape(-1), 2))
print('Scale된 test_array 데이터:', np.round(test_scaled.reshape(-1), 2))
```

줄	코드	상세 설명
1~2	scaler = MinMaxScaler() / scaler.fit(train_array)	스케일러를 새로 만들고 학습 데이터로만 fit() 합니다. 기준: 최솟값 0, 최댓값 10.
3	train_scaled = scaler.transform(train_array)	학습 데이터를 1/10 스케일로 변환합니다.
8	test_scaled = scaler.transform(test_array)	핵심입니다. fit() 을 다시 호출하지 않고 transform() 만 호출합니다. 학습 데이터에서 만든 1/10 기준이 그대로 적용되어 원본 5가 0.5로 변환됩니다.
10	print(...)	테스트 결과가 0, 0.1, 0.2, 0.3, 0.4, 0.5로 학습 데이터와 완벽히 동일한 척도를 갖게 되었습니다.

▶ 실행 결과

```
원본 train_array 데이터: [ 0  1  2  3  4  5  6  7  8  9 10]
Scale된 train_array 데이터: [0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]

원본 test_array 데이터: [0 1 2 3 4 5]
Scale된 test_array 데이터: [0.  0.1 0.2 0.3 0.4 0.5]
```

■ 반드시 기억할 규칙

1. 가능하다면 전체 데이터를 나누기 전에 스케일링을 먼저 수행합니다.
2. 그것이 어렵다면, 테스트 데이터에는 절대 fit() 이나 fit_transform() 을 사용하지 말고 학습 데이터로 fit한 객체의 transform() 만 호출합니다.
3. 이 원칙은 모든 전처리 클래스(스케일러, 인코더, Imputer 등)에 동일하게 적용됩니다.

실무에서는 Pipeline 을 사용하면 이 규칙이 자동으로 지켜지므로 실수를 원천적으로 방지할 수 있습니다.

```
pipe = make_pipeline(MinMaxScaler(), DecisionTreeClassifier())
```

5. 핵심 요약 및 머신러닝 활용 포인트

5.1 핵심 API 요약표

구분	API	기능	주의 사항
인코딩	<code>LabelEncoder</code>	문자열 → 코드 숫자	크기 관계 발생. 선형 모델에는 부적합 <code>classes_</code> , <code>inverse_transform()</code> 활용
	<code>OneHotEncoder</code>	고유값 수만큼 컬럼 생성	2차원 입력 필요 , 결과는 희소 행렬 <code>sparse_output=False</code> 로 밀집 행렬 반환
	<code>pd.get_dummies()</code>	판다스 원-핫 인코딩	한 줄로 처리, 컬럼명 자동 생성 pandas 2.x부터 bool 반환
스케일링	<code>StandardScaler</code>	평균 0, 분산 1로 표준화	정규 분포 가정 알고리즘에 유리
	<code>MinMaxScaler</code>	0~1 범위로 정규화	이상치에 취약
	<code>RobustScaler</code>	중앙값·IQR 기준 변환	이상치가 많을 때 권장
공통 메서드	<code>fit()</code>	변환 기준 학습	학습 데이터에만 호출
	<code>transform()</code>	실제 변환 수행	학습·테스트 데이터 모두에 호출
	<code>fit_transform()</code>	두 작업을 한 번에	테스트 데이터에는 사용 금지

5.2 인코딩 방식 선택 가이드

상황	권장 방식	이유
결정 트리 계열 모델	레이블 인코딩	숫자 크기를 해석하지 않아 문제 없음. 차원도 늘지 않음
선형 회귀·로지스틱·SVM	원-핫 인코딩	레이블 인코딩의 크기 관계가 가중치를 왜곡함
고유값이 매우 많은 피처	상위 값 묶기 후 인코딩	원-핫은 차원 폭발, 레이블은 의미 왜곡
순서가 있는 카테고리 (초급<중급<고급)	레이블/순서 인코딩	크기 관계가 실제로 의미가 있는 경우

5.3 머신러닝 활용 포인트

- **전처리 순서** : 결측치 처리 → 불필요 피처 제거 → 인코딩 → 스케일링 순으로 진행하는 것이 일반적입니다.
- **알고리즘에 맞는 전처리** : 트리 계열은 스케일링이 거의 불필요하지만, 선형·거리 기반 모델은 필수입니다. 무조건 다 적용하기보다 **모델 특성을 고려**하세요.
- **데이터 누수(Data Leakage) 방지** : 테스트 데이터의 통계 정보가 전처리 과정에 스며들면 성능이 부풀려집니다. `fit` 은 반드시 학습 데이터에만 적용하세요.
- **Pipeline 활용** : 전처리와 모델을 하나로 묶으면 교차 검증에서도 폴드마다 올바르게 fit/transform이 적용되어 **누수를 자동으로 방지**합니다.
- **변환 후 검증** : 스케일링 후 평균·분산·최솟값·최댓값을 직접 출력해 **의도대로 변환되었는지 확인**하는 습관을 들이세요.

⚠ 안정화 버전 참고

- **scikit-learn 1.2+** : `OneHotEncoder` 의 `sparse` → `sparse_output` 으로 변경(1.4에서 구 파라미터 제거).
- **pandas 2.0+** : `get_dummies( )` 결과가 0/1 정수에서 **True/False(bool)**로 변경. `dtype=int` 로 기존 동작 유지 가능.
- `LabelEncoder` 는 원래 **레이블(y) 전용**으로 설계된 클래스입니다. 피처(X)의 카테고리 인코딩에는 `OrdinalEncoder` 를 쓰는 것이 사이킷런의 공식 권장 사항입니다. 다만 교재와 실무에서는 관행적으로 피처에도 널리 사용됩니다.